

Dateiauswertung

GLEICH**.git/hooks/fsmonitor-watchman.sample**

Branch 1	main
Branch 2	current
Datei	.git/hooks/fsmonitor-watchman.sample
Typ	Datei

Bewertung

SHA-256 identisch.

Dateiinhalt / Vergleich

1	#!/usr/bin/perl
2	
3	use strict;
4	use warnings;
5	use IPC::Open2;
6	
7	# An example hook script to integrate Watchman
8	# (https://facebook.github.io/watchman/) with git to speed up detecting
9	# new and modified files.
10	#
11	# The hook is passed a version (currently 2) and last update token
12	# formatted as a string and outputs to stdout a new update token and
13	# all files that have been modified since the update token. Paths must
14	# be relative to the root of the working tree and separated by a single NUL.
15	#
16	# To enable this hook, rename this file to "query-watchman" and set
17	# 'git config core.fsmonitor .git/hooks/query-watchman'
18	#
19	my (\$version, \$last_update_token) = @ARGV;
20	
21	# Uncomment for debugging
22	# print STDERR "\$0 \$version \$last_update_token\n";
23	
24	# Check the hook interface version
25	if (\$version ne 2) {
26	die "Unsupported query-fsmonitor hook version '\$version'.\n" .
27	"Falling back to scanning...\n";
28	}
29	
30	my \$git_work_tree = get_working_dir();

31	
32	my \$json_pkg;
33	eval {
34	require JSON::XS;
35	\$json_pkg = "JSON::XS";
36	1;
37	} or do {
38	require JSON::PP;
39	\$json_pkg = "JSON::PP";
40	};
41	
42	launch_watchman();
43	
44	sub launch_watchman {
45	my \$o = watchman_query();
46	if (is_work_tree_watched(\$o)) {
47	output_result(\$o->{clock}, @{\$o->{files}});
48	}
49	}
50	
51	sub output_result {
52	my (\$clockid, @files) = @_;
53	
54	# Uncomment for debugging watchman output
55	# open (my \$fh, ">", ".git/watchman-output.out");
56	# binmode \$fh, ":utf8";
57	# print \$fh "\$clockid\n@files\n";
58	# close \$fh;
59	
60	binmode STDOUT, ":utf8";
61	print \$clockid;
62	print "\0";
63	local \$, = "\0";
64	print @files;
65	}
66	
67	sub watchman_clock {
68	my \$response = qx/watchman clock "\$git_work_tree"/;
69	die "Failed to get clock id on '\$git_work_tree'.\n" .
70	"Falling back to scanning...\n" if \$? != 0;
71	
72	return \$json_pkg->new->utf8->decode(\$response);
73	}
74	
75	sub watchman_query {
76	my \$pid = open2(*CHLD_OUT, *CHLD_IN, 'watchman -j --no-pretty')
77	or die "open2() failed: \$!\n" .
78	"Falling back to scanning...\n";
79	
80	# In the query expression below we're asking for names of files that
81	# changed since \$last_update_token but not from the .git folder.

82	#
83	# To accomplish this, we're using the "since" generator to use the
84	# recency index to select candidate nodes and "fields" to limit the
85	# output to file names only. Then we're using the "expression" term to
86	# further constrain the results.
87	my \$last_update_line = "";
88	if (substr(\$last_update_token, 0, 1) eq "c") {
89	\$last_update_token = "\"\$last_update_token\"";
90	\$last_update_line = qq[\n"since": \$last_update_token,];
91	}
92	my \$query = <<" END";
93	["query", "\$git_work_tree", {\$last_update_line
94	"fields": ["name"],
95	"expression": ["not", ["dirname", ".git"]]
96	}]
97	END
98	
99	# Uncomment for debugging the watchman query
100	# open (my \$fh, ">", ".git/watchman-query.json");
101	# print \$fh \$query;
102	# close \$fh;
103	
104	print CHLD_IN \$query;
105	close CHLD_IN;
106	my \$response = do {local \$/; <CHLD_OUT>};
107	
108	# Uncomment for debugging the watch response
109	# open (\$fh, ">", ".git/watchman-response.json");
110	# print \$fh \$response;
111	# close \$fh;
112	
113	die "Watchman: command returned no output.\n" .
114	"Falling back to scanning...\n" if \$response eq "";
115	die "Watchman: command returned invalid output: \$response\n" .
116	"Falling back to scanning...\n" unless \$response =~ /\n/;
117	
118	return \$json_pkg->new->utf8->decode(\$response);
119	}
120	
121	sub is_work_tree_watched {
122	my (\$output) = @_;
123	my \$error = \$output->{error};
124	if (\$error and \$error =~ m/unable to resolve root .* directory (.*) is not watched/) {
125	my \$response = qx/watchman watch "\$git_work_tree"/;
126	die "Failed to make watchman watch '\$git_work_tree'.\n" .
127	"Falling back to scanning...\n" if \$? != 0;
128	\$output = \$json_pkg->new->utf8->decode(\$response);
129	\$error = \$output->{error};
130	die "Watchman: \$error.\n" .
131	"Falling back to scanning...\n" if \$error;
132	

133	# Uncomment for debugging watchman output
134	# open (my \$fh, ">", ".git/watchman-output.out");
135	# close \$fh;
136	
137	# Watchman will always return all files on the first query so
138	# return the fast "everything is dirty" flag to git and do the
139	# Watchman query just to get it over with now so we won't pay
140	# the cost in git to look up each individual file.
141	my \$o = watchman_clock();
142	\$error = \$o->{error};
143	
144	die "Watchman: \$error.\n" .
145	"Falling back to scanning...\n" if \$error;
146	
147	output_result(\$o->{clock}, ("/"));
148	return 0;
149	}
150	
151	die "Watchman: \$error.\n" .
152	"Falling back to scanning...\n" if \$error;
153	
154	return 1;
155	}
156	
157	sub get_working_dir {
158	my \$working_dir;
159	if (\$^O =~ 'msys' \$^O =~ 'cygwin') {
160	\$working_dir = Win32::GetCwd();
161	\$working_dir =~ tr/\\\/\//;
162	} else {
163	require Cwd;
164	\$working_dir = Cwd::cwd();
165	}
166	
167	return \$working_dir;
168	}